In [1]:

```
# A lambda function is a small anonymous function ,
# which can take any no of args but can only have a one expression
```

In [1]:

```
#add function
add=lambda a,b:a+b
add(3,8)
```

Out[1]:

```
11
```

In [2]:

```
add=lambda x:x+100
add(5)
```

Out[2]:

```
105
```

In [3]:

```
(lambda a,b:a*b)(3,5) #iief
```

Out[3]:

```
15
```

In [4]:

```
#mul
mul=lambda a,b:a*b
mul(b=3,a=8)
```

Out[4]:

```
24
```

In [7]:

```
#mul
mul=lambda a=5,b=6:a*b
mul()
```

Out[7]:

```
30
```

In [8]:

```
add=lambda *args:sum(args)
add(1,2,3,4,5)
```

Out[8]:

```
15
```

In [1]:

```
op=lambda x,f:x+f(x)
op(10,lambda x:x*x)
```

Out[1]:

```
110
```

In [13]:

```
(lambda x:(x%2 and 'odd' or 'even'))(40)
```

Out[13]:

```
'even'
```

In [14]:

```
sub=lambda s:s in 'abcdefg'
sub('a')
```

Out[14]:

```
True
```

In [17]:

```
num=[10,3,23,43,2,1]
grt=list(filter(lambda num:num>30,num))
grt
```

Out[17]:

```
[43]
```

In [19]:

```
div=list(filter(lambda num:(num%2!=0),num))
div
```

Out[19]:

```
[3, 23, 43, 1]
```

In [20]:

```
mul=list(map(lambda x:x*2,num))
mul
```

Out[20]:

```
[20, 6, 46, 86, 4, 2]
```

In [21]:

```
cube=list(map(lambda x:x**3,num))
cube
```

Out[21]:

```
[1000, 27, 12167, 79507, 8, 1]
```

In [24]:

```
from functools import reduce
s=reduce((lambda x,y:x+y),num)
s
```

Out[24]:

```
82
```

In [3]:

```
def myfun(n):
    return lambda a:a*n
doubler=myfun(2)
tripler=myfun(3)
print(doubler(11))
print(tripler(11))
```

```
22
33
```

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]: